Instituto Superior
de Engenharia
Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

Licenciatura de Engenharia Informática

Arquiteturas Móveis - 2025/2026

Trabalho Prático

Flutter

Autores / Authors

José Tôco **2023141186**

Paulo Soares **2023140277**

Instituto Superior
de Engenharia
Politécnico de Coimbra

1 Índice

2 Introdução 2
3 Arquitetura e Organização do Projeto 2
3.1 Estrutura de Ficheiros 2
3.2 Estrutura de Camadas 3
3.3 Fluxo de Inicialização 3
4 Detalhes de Implementação 3
4.1 Widgets: Stateless vs. Stateful 4
4.2 Gestão de Estado (State Management) 4
4.3 Arquitetura de Dados (DataManager) 5
4.4 Navegação 7
4.5 Interface e UI/UX (Material Design 3) 10
4.6 Integração de Serviços Externos (Meteorologia) 11
4.7 Programação Assíncrona e Performance 12
5 Desafios e Soluções 13
5.1 Acesso Global aos Dados (DataManager) 13
5.2 Navegação Complexa e Preservação de Estado 14
5.3 Reatividade e Gestão de Estado 14
5.4 UI Adaptável e Experiência de Utilizador (UI/UX) 14
5.5 Fluidez e Performance (Assincronismo) 14
6 Análise Crítica e Melhorias Futuras 15
7 Conclusão 16

2 INTRODUÇÃO

O presente relatório detalha o desenvolvimento de uma aplicação móvel para um Guia Turístico da cidade de Coimbra, concebida no âmbito da unidade curricular de **Arquiteturas Móveis**. O projeto explora as capacidades da framework **Flutter** e da linguagem **Dart** para a criação de interfaces modernas, eficientes e multiplataforma.

O objetivo central foi projetar e implementar uma arquitetura de software robusta que suporte funcionalidades essenciais de uma aplicação de turismo: consulta dinâmica de pontos de interesse organizados por categorias, visualização detalhada de informação, gestão personalizada de favoritos e integração de dados meteorológicos em tempo real.

Para além da vertente funcional, o projeto foca-se no rigor técnico da implementação, abordando desafios fundamentais como a persistência de dados local, a gestão de estado manual sem dependências externas pesadas, a navegação estruturada por abas e a otimização de performance através de processos assíncronos. A solução aqui descrita reflete uma abordagem focada na modularidade e na separação de responsabilidades, garantindo a escalabilidade e a facilidade de manutenção do código.

3 ARQUITETURA E ORGANIZAÇÃO DO PROJETO

A arquitetura do projeto foi desenhada para promover a **separação de responsabilidades**, garantindo que a interface gráfica não contenha lógica de negócio complexa. O sistema organiza-se em camadas lógicas distintas, refletidas na estrutura de pastas:

3.1 Estrutura de Ficheiros

A organização do projeto segue as convenções da framework Flutter, estruturando os recursos e o código fonte de forma modular para facilitar a manutenção. Abaixo é apresentada a árvore de diretórios principal (focada nas pastas `lib` e `assets`):

3.2 Estrutura de Camadas

- **Camada de Modelos de Dados (`lib/models`):** Contém as definições fundamentais dos dados. As classes `POI` e `WeatherInfo` são objetos puros responsáveis apenas por representar a informação e realizar a conversão de/para JSON, sem dependências da interface.
- **Camada de Serviços (`lib/services`):** Centraliza a lógica da aplicação.
  - `DataManager`: Atua como um repositório central e mediador. Isola a origem dos dados (ficheiros locais ou memória) da interface.
  - `WeatherService`: Encapsula a comunicação com a API externa, tratando pedidos HTTP e erros de rede.
- **Camada de Apresentação (`lib/screens` e `lib/widgets`):** Focada exclusivamente na renderização da interface e interação com o utilizador.
  - `screens`: Representam páginas completas e gerem o ciclo de vida da navegação.
  - `widgets`: Componentes visuais reutilizáveis e modulares (ex: cartões).

3.3 Fluxo de Inicialização

A entrada da aplicação (`main.dart`) foi estruturada para garantir a integridade dos dados antes da renderização da interface. O método `main()` é assíncrono e executa os seguintes passos críticos:

1. **Vínculo com o Motor:** Invoca *`WidgetsFlutterBinding.ensureInitialized()`* para garantir que a camada de *binding* do Flutter com o sistema operativo está pronta, um pré-requisito para utilizar plugins nativos como o *`SharedPreferences`*.
2. **Pré-carregamento de Dados:** Inicializa o Singleton `DataManager` e aguarda (*`await`*) o carregamento do ficheiro JSON e das preferências do utilizador.
3. **Arranque da UI:** Apenas após a conclusão destas tarefas é chamado o *`runApp()`*. Esta abordagem previne que a aplicação inicie com estados inconsistentes ou dados nulos, eliminando a necessidade de lógica complexa de tratamento de "dados vazios" nos ecrãs iniciais.

4 DETALHES DE IMPLEMENTAÇÃO

Nesta secção detalham-se as opções técnicas tomadas para resolver os requisitos funcionais, ilustradas com excertos de código relevantes do projeto.

4.1 Widgets: Stateless vs. Stateful

A escolha entre widgets foi criteriosa, baseada na necessidade de manipulação do ciclo de vida:

- **Stateless Widgets (Eficiência):** Privilegiou-se o uso destes widgets (ex: `CategoryCard`, `POICard`) para componentes que dependem apenas da sua configuração inicial.
- **Stateful Widgets (Ciclo de Vida):** Essenciais para ecrãs com lógica temporal ou reativa. No `HomeScreen`, o método *`initState()`* inicia o temporizador da meteorologia e o *`dispose()`* cancela-o para prevenir fugas de memória:

```dart
// Excertos de lib/screens/HomeScreen.dart
@override
void initState() {
  super.initState();
  _fetchWeather();
  // Update weather every 15 minutes
  _weatherTimer = Timer.periodic(const Duration(minutes: 15), (timer) {
    _fetchWeather();
  });
}

@override
void dispose() {
  _weatherTimer?.cancel();
  super.dispose();
}
```

4.2 Gestão de Estado (State Management)

Sem recorrer a bibliotecas externas complexas, a aplicação utiliza uma gestão de estado manual baseada no método *`setState()`*. A aplicação deste método divide-se em três cenários distintos:

1. **Atualização de Estado Local:** Utilizada quando é necessário atualizar variáveis internas do widget e refletir esses dados na UI. No exemplo abaixo, as variáveis de meteorologia são atribuídas dentro do bloco *`setState`*, garantindo que a framework deteta a alteração:

```dart
// Excerto de lib/screens/HomeScreen.dart
setState(() {
  _weather = weather;
  _loadingWeather = false;
});
```

2. **Reatividade Local:** Garante feedback visual imediato após uma interação do utilizador. Ao clicar no botão de favoritos, a operação assíncrona é aguardada e, de seguida, o `setState` vazio força a reconstrução do ícone (de vazio para cheio ou vice-versa):

```dart
// Excerto de lib/screens/POIDetailsScreen.dart
onPressed: () async {
  await _dataManager.toggleFavorite(widget.poi.id);
  setState(() {}); // Reconstrói o widget para atualizar o ícone
},
```

3. **Sincronização no Retorno:** Assegura a consistência entre ecrãs. Ao navegar de volta de um ecrã de detalhe, é crucial atualizar a lista anterior (pois um favorito pode ter sido removido). O método `.then()` do `Navigator` captura o momento do regresso e invoca o `setState`:

```dart
// Excerto de lib/screens/POIsListScreen.dart
onTap: () {
  Navigator.of(context, rootNavigator: true).push(
    MaterialPageRoute(builder: (context) => POIDetailsScreen(poi: poi)),
  ).then((_) {
    // Força a reconstrução da lista ao voltar do ecrã de detalhe
    setState(() {});
  });
},
```

4.3 Arquitetura de Dados (DataManager)

A classe `DataManager` atua como a "Fonte Única de Verdade" da aplicação. A sua implementação é complexa e cobre várias responsabilidades críticas:

A. Padrão Singleton Garante que todos os ecrãs acedem exatamente à mesma instância de dados em memória, evitando dessincronização entre vistas (ex: lista e detalhe).

```dart
class DataManager {
  static final DataManager _instance = DataManager._internal();
  factory DataManager() => _instance;
  DataManager._internal();
  // ...
}
```

B. Inicialização e Parsing JSON O método `loadData` é responsável por orquestrar o arranque da aplicação. Lê o ficheiro `assets/data/poi.json` usando `rootBundle`, descodifica a estrutura JSON e itera sobre as categorias e pontos para preencher as listas em memória (`_allPois` e `_categories`).

```dart
// Excerto de lib/services/DataManager.dart
Future<void> loadData() async {
  final String response = await rootBundle.loadString('assets/data/poi.json');
  final data = await json.decode(response);

  // Lógica de parsing iterando sobre categorias e pontos
  for (var catJson in data['categories']) {
     // ... criação de objetos POI e preenchimento de listas
  }

  // Carrega favoritos do disco após processar o JSON
  await _loadFavorites();
}
```

C. Persistência de Favoritos A gestão de favoritos utiliza um *`Set<int>`* para garantir acessos rápidos (complexidade O(1)) e evitar duplicados. A persistência é assegurada pelo `SharedPreferences`, convertendo a lista de IDs inteiros para Strings antes de guardar no disco.

```dart
// Excerto de lib/services/DataManager.dart
Future<void> toggleFavorite(int poiId) async {
  // Atualiza em memória
  if (_favorites.contains(poiId)) {
    _favorites.remove(poiId);
  } else {
    _favorites.add(poiId);
  }

  // Persiste no disco
  final prefs = await SharedPreferences.getInstance();
  await prefs.setStringList('favorites', _favorites.map((id) => id.toString()).toList());
}
```

D. Encapsulamento e Filtragem O `DataManager` expõe métodos utilitários que simplificam a lógica nos *Widgets*, como `getPoisByCategory(String category)` e `getFavorites()`, centralizando a lógica de filtragem.

E. Tratamento de Exceções e Robustez Para evitar que a aplicação encerre abruptamente caso o ficheiro JSON esteja corrompido ou inacessível, todo o processo de carregamento está protegido por um bloco *`try-catch-finally`*. O bloco *`finally`* garante que o estado de carregamento (`_isLoading`) é definido como `false` mesmo em caso de erro, permitindo que a aplicação saia do estado de "loading" e apresente uma interface (mesmo que vazia), em vez de ficar presa num loop infinito de carregamento.

```dart
// Excerto de lib/services/DataManager.dart
try {
  // Tentativa de carregar e fazer parse do JSON
  final String response = await rootBundle.loadString('assets/data/poi.json');
  // ...
} catch (e) {
  if (kDebugMode) {
    print('Error loading POIs: $e'); // Log do erro para depuração
  }
} finally {
  _isLoading = false; // Garante que a UI desbloqueia
}
```

4.4 Navegação

A navegação da aplicação segue um modelo hierárquica. O ponto de entrada é o **Ecrã Inicial (Landing Page)**, que utiliza o *`Navigator.push`* para empilhar a `MainScreen` na pilha global. Esta transição permite que o ecrã inicial permaneça na base da pilha, sendo restaurado automaticamente quando o utilizador decide sair da área funcional da aplicação. No interior da `MainScreen`, a navegação expande-se para um modelo híbrido de fluxos lineares e abas aninhadas.

A. **Estratégia de Roteamento Explícito** A aplicação utiliza **Navegação Explícita** (através de *`Navigator.push`* e *`MaterialPageRoute`*) em detrimento de rotas nomeadas (*Named Routes*).

   – **Passagem de Objetos:** Esta escolha permite passar instâncias completas das classes de modelo (ex: o objeto `POI`) diretamente para o construtor do ecrã de destino. Isto é tecnicamente superior às rotas nomeadas para este caso de uso, pois evita a necessidade de extrair argumentos de forma genérica e garante a integridade dos dados através do sistema de tipos do Dart.

   – **Flexibilidade:** Facilita a implementação da navegação aninhada nos múltiplos `Navigators` da `MainScreen`, permitindo criar rotas dinâmicas de forma mais intuitiva e segura.

B. **Navegação Aninhada e o papel do Map (`_navigatorKeys`)** Em vez de utilizar um único `Navigator` para toda a aplicação, a `MainScreen` implementa um padrão de **Navegação Aninhada**. Cada aba ("Categorias" e "Favoritos") possui o seu próprio widget `Navigator`, gerido através de um *Map* de chaves globais (`_navigatorKeys`).

- **Benefício:** Isto permite que o histórico de navegação de cada aba seja independente. O utilizador pode navegar profundamente na hierarquia de "Categorias", mudar para "Favoritos", e ao voltar a "Categorias" encontrar o ecrã exato onde estava, em vez de ser forçado a voltar ao início.



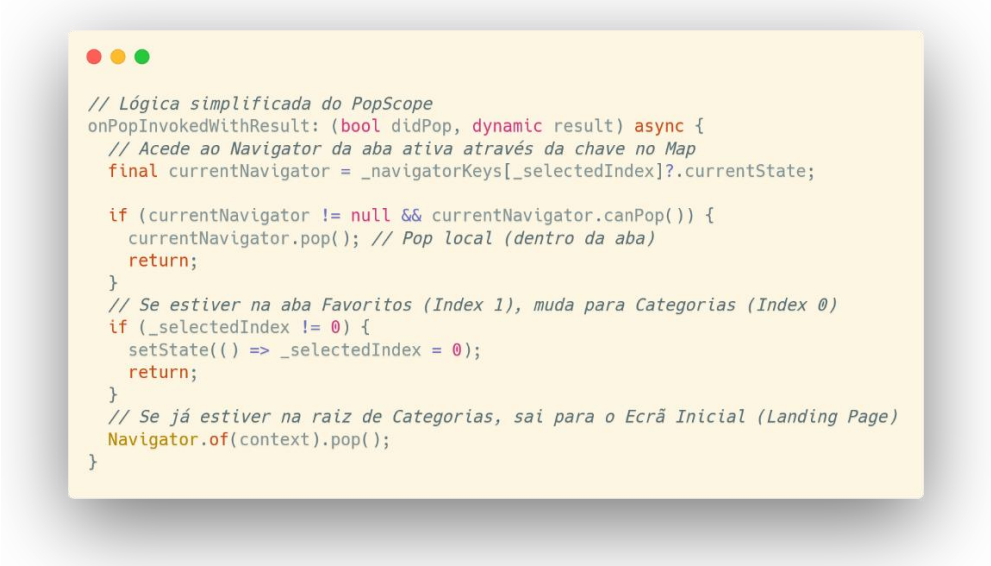
```dart
// Excerto de lib/screens/MainScreen.dart
children: [
  // Tab 0: Navigator independente para o fluxo de Categorias
  Navigator(
    key: _navigatorKeys[0],
    onGenerateRoute: (settings) => MaterialPageRoute(builder: (_) => const CategoriesScreen()),
  ),
  // Tab 1: Navigator independente para o fluxo de Favoritos
  Navigator(
    key: _navigatorKeys[1],
    onGenerateRoute: (settings) => MaterialPageRoute(builder: (_) => FavoritesScreen(...)),
  ),
],
```

C. **Preservação de Estado com `IndexedStack`** Para evitar a destruição e reconstrução dos ecrãs ao alternar abas, utilizou-se o widget ***`IndexedStack`***.

- **Diferenciação Técnica:** Ao contrário de uma troca de widgets comum (que limpa a memória e o estado), o `IndexedStack` mantém todos os seus filhos "vivos" na árvore de elementos, apenas alterando a visibilidade. Isto garante que a posição do scroll nas listas e o histórico de navegação interna de cada aba sejam preservados intactos, proporcionando uma experiência de utilizador fluida e instantânea.

D. **Gestão do Botão "Voltar" (PopScope)** Para garantir uma experiência nativa Android correta, o widget ***`PopScope`*** interceta o botão físico de retrocesso e aplica uma lógica hierárquica recorrendo às chaves guardadas no Map:

4. **Nível Local:** Primeiro, tenta retroceder dentro da própria aba ativa (se houver histórico, como um detalhe aberto).
5. **Navegação entre Abas:** Se não houver histórico, verifica se estamos na aba "Favoritos" e, nesse caso, redireciona para a aba **Categorias**.
6. **Saída da Aplicação:** Apenas se o utilizador estiver na raiz da aba Categorias é que a aplicação permite sair da `MainScreen`, retornando ao **Ecrã Inicial**. No **Ecrã Inicial**, embora não exista um botão físico de "Voltar" na interface, a utilização do botão de retrocesso do sistema causará o encerramento da aplicação, respeitando o ciclo de vida padrão do Android.



```dart
// Lógica simplificada do PopScope
onPopInvokedWithResult: (bool didPop, dynamic result) async {
  // Acede ao Navigator da aba ativa através da chave no Map
  final currentNavigator = _navigatorKeys[_selectedIndex]?.currentState;

  if (currentNavigator != null && currentNavigator.canPop()) {
    currentNavigator.pop(); // Pop local (dentro da aba)
    return;
  }
  // Se estiver na aba Favoritos (Index 1), muda para Categorias (Index 0)
  if (_selectedIndex != 0) {
    setState(() => _selectedIndex = 0);
    return;
  }
  // Se já estiver na raiz de Categorias, sai para o Ecrã Inicial (Landing Page)
  Navigator.of(context).pop();
}
```

E. **Navegação para Detalhes (Root Navigator)** Ao clicar num Ponto de Interesse, a aplicação navega para o ecrã `POIDetailsScreen`. Esta transição é executada invocando o Navigator com a propriedade `rootNavigator: true`.

- **Experiência Imersiva:** O uso do Navigator de raiz permite que o ecrã de detalhes seja empilhado sobre a `MainScreen` na totalidade, cobrindo inclusive a barra de abas inferior (*`BottomNavigationBar`*). Isto proporciona uma área de visualização maior e uma experiência de leitura mais focada no conteúdo.
- **Eficiência e Passagem de Dados:** O objeto `POI` completo é passado diretamente através do construtor. Isto evita a necessidade de realizar novas pesquisas na base de dados (`DataManager`) ou novos pedidos de rede, tornando a transição instantânea e garantindo a integridade da informação exibida.



```dart
// Exemplo de navegação para a raiz
Navigator.of(context, rootNavigator: true).push(
  MaterialPageRoute(builder: (context) => POIDetailsScreen(poi: poi)),
);
```

4.5 Interface e UI/UX (Material Design 3)

A interface da aplicação foi construída utilizando os princípios do **Material Design 3**, focando-se na clareza visual e na eficiência do layout através de uma composição rica de widgets:

- **Tema Global e Estilo:** O estilo visual é centralizado no *`ThemeData`* (em `main.dart`), utilizando o método *`ColorScheme.fromSeed`* para gerar automaticamente uma paleta de cores coesa baseada num tom primário (Deep Purple). Isto garante consistência visual em todos os ecrãs sem necessidade de definir cores manualmente em cada widget.
- **Tipografia e Hierarquia Visual:** Em vez de utilizar tamanhos de fonte fixos, a aplicação recorre à hierarquia tipográfica do Material 3 (ex: *`headlineSmall`*, *`titleLarge`*, *`bodyMedium`*). O acesso a estes estilos através do *`Theme.of(context).textTheme`* garante que o texto se adapta proporcionalmente às definições do tema, mantendo a legibilidade e o equilíbrio visual em todos os ecrãs.
- **Estrutura Base (Scaffold & AppBar):** Todos os ecrãs utilizam o widget *`Scaffold`* para gerir a estrutura básica (barra superior, corpo e navegação). A `AppBar` foi configurada de forma global no tema (`main.dart`) para manter a consistência, utilizando títulos centrados e elevação suave.
- **Adaptabilidade (SafeArea):** Para garantir que o conteúdo não é obstruído por elementos do sistema (como o "notch" da câmara ou barras de navegação), utilizou-se o widget `SafeArea` nos ecrãs principais, como o `HomeScreen`.
- **Listagens Eficientes (ListView.builder):** Nas listas de categorias e pontos de interesse, utilizou-se o *`ListView.builder`*. Esta abordagem é fundamental para a performance, pois renderiza os itens "on-demand" (apenas os que estão visíveis), otimizando o consumo de memória em listas longas.
- **Content Containers (Card & ListTile):** Os Pontos de Interesse são encapsulados em `Cards` com cantos arredondados e elevação controlada. No interior, o widget *`ListTile`* organiza de forma limpa a imagem à esquerda (*`leading`*), o texto central (*`title`* e *`subtitle`*) e as ações à direita (*`trailing`*).
- **Composição de Layouts (Stack, Column, Row):**
  - O ecrã inicial (`HomeScreen`) utiliza um *`Stack`* para sobrepor a imagem de fundo, um gradiente escuro (criado com *`BoxDecoration`* e *`LinearGradient`*) e o conteúdo textual.
  - As informações detalhadas em `POIDetailsScreen` são organizadas em *`Columns`* dentro de um *`SingleChildScrollView`* para garantir a legibilidade em ecrãs de menores dimensões.

- **Feedback Visual e Interação:**
  - **Indicadores de Estado:** Utiliza-se o *`CircularProgressIndicator`* para informar o utilizador durante operações de carregamento (ex: leitura do JSON ou pedido de meteorologia).
  - **Interatividade:** Os cartões de categoria utilizam *`InkWell`* para fornecer feedback visual de toque. O botão principal na Home utiliza *`WidgetStateProperty`* para ajustar a sua elevação e cor de forma reativa consoante o estado (pressionado ou em repouso).
- **Tratamento de Assets e Erros:** O widget *`Image.asset`* foi robustecido com um *`errorBuilder`* que exibe um ícone de substituição caso a imagem não exista no sistema de ficheiros, prevenindo buracos no layout.

```dart
// Exemplo de tratamento de erro de imagem
errorBuilder: (context, error, stackTrace) => Container(
  height: 250,
  color: Colors.grey[300],
  child: const Icon(Icons.image_not_supported, size: 100),
),
```

4.6 Integração de Serviços Externos (Meteorologia)

A funcionalidade meteorológica demonstra a capacidade da aplicação em consumir serviços externos e gerir tarefas temporizadas em segundo plano.

A. Consumo de API REST (Pacote `http`) O serviço `WeatherService` encapsula a comunicação com a API da Open-Meteo. Utiliza o pacote `http` para realizar pedidos assíncronos (`GET`), garantindo que a aplicação permanece responsiva. A construção do URL é feita dinamicamente através da classe `Uri`, passando as coordenadas geográficas de Coimbra.



```dart
// Excerto de lib/services/WeatherService.dart
Future<WeatherInfo?> fetchWeather() async {
  final url = Uri.parse('$_baseUrl?latitude=$_lat&longitude=$_lon&current_weather=true');

  try {
    final response = await http.get(url);
    if (response.statusCode == 200) {
      return WeatherInfo.fromJson(json.decode(response.body));
    }
  } catch (e) {
    // Tratamento de erros de rede
  }
  return null;
}
```

B. **Processamento e Mapeamento de Dados** A resposta JSON da API é processada pelo modelo `WeatherInfo`. Um detalhe técnico relevante é a conversão dos códigos meteorológicos (WMO codes) fornecidos pela API para descrições textuais e ícones visuais representativos, centralizando esta lógica de apresentação no modelo de dados.

C. **Atualização Periódica e Ciclo de Vida (Timer)** Para assegurar a atualidade da informação sem intervenção do utilizador, foi implementado um **Timer Periódico** no `HomeScreen`.

- **Frequência:** Definida para 15 minutos, equilibrando a frescura dos dados com a economia de bateria e tráfego de rede.
- **Gestão de Memória:** O Timer é iniciado no `initState` e, obrigatoriamente, cancelado no `dispose`. Esta prática é crucial para evitar fugas de memória e execuções de código em ecrãs que já foram removidos da memória.

4.7 Programação Assíncrona e Performance

O Dart opera num modelo de execução *Single Thread* com um **Event Loop**, o que significa que qualquer operação pesada que bloqueie a execução sequencial irá congelar a interface. Para mitigar isto, o projeto utiliza `Future` e `async/await` para delegar operações de I/O (Input/Output).

- **Modelo Concorrente:** Ao marcar uma função como `async` e utilizar `await`, o código "pausa" a execução da função atual e retorna o controlo ao *Event Loop*, permitindo que a aplicação continue a processar toques e a redesenhar a interface enquanto aguarda o resultado.
- **Sequencialidade Lógica:** O `await` permite escrever código assíncrono que se lê de forma síncrona (sequencial), facilitando o tratamento de erros (`try-catch`) e a manutenção da lógica de negócio, como visto no carregamento inicial de dados no `DataManager`.

- **Isolamento de I/O:** Todas as operações críticas, como a leitura do ficheiro `poi.json`, pedidos HTTP à Open-Meteo e escrita nas `SharedPreferences`, são executadas de forma assíncrona, garantindo uma experiência de utilização fluida e sem interrupções.

```dart
// Exemplo de carregamento assíncrono no DataManager
Future<void> loadData() async {
  _isLoading = true;
  try {
    // A espera (await) não bloqueia a UI Thread
    final String response = await rootBundle.loadString('assets/data/poi.json');
    // ...
    await _loadFavorites();
  } finally {
    _isLoading = false; // Garante estado consistente
  }
}
```

```dart
// Exemplo de persistência assíncrona em background
Future<void> toggleFavorite(int poiId) async {
  // ... lógica de atualização da lista em memória ...

  final prefs = await SharedPreferences.getInstance();
  // Operação de I/O (escrita) é aguardada sem congelar a app
  await prefs.setStringList('favorites', _favorites.map((id) => id.toString()).toList());
}
```

5 DESAFIOS E SOLUÇÕES

Durante o desenvolvimento, a equipa enfrentou desafios técnicos que exigiram a adoção de padrões e estratégias específicas:

5.1 Acesso Global aos Dados (DataManager)

Desafio: A necessidade de partilhar a lista de POIs e o estado dos favoritos entre múltiplos ecrãs (Lista, Detalhe, Favoritos) criaria uma complexidade de passagem de parâmetros insustentável.

Solução: Implementação do padrão **Singleton** na classe `DataManager`. Isto criou uma instância única e globalmente acessível, centralizando o acesso aos dados da aplicação.

5.2 Navegação Complexa e Preservação de Estado

Desafio: Ao utilizar uma *`BottomNavigationBar`* padrão, a troca de abas destruía o estado dos widgets, fazendo com que o utilizador perdesse a sua posição de scroll ou a navegação interna numa categoria.

Solução: Combinação de **Navegação Aninhada** (um `Navigator` por aba) com o widget ***`IndexedStack`***. Esta arquitetura permitiu manter múltiplos históricos de navegação ativos em memória simultaneamente, proporcionando uma experiência de multitarefa fluida.

5.3 Reatividade e Gestão de Estado

Desafio: Garantir a atualização da UI em resposta a eventos globais (ex: remover favorito noutro ecrã) sem complicar a arquitetura com dependências excessivas.

Solução: Opção pela **simplicidade** com o uso estratégico do método ***`setState()`*** e callbacks de retorno do `Navigator` (*`.then()`*). Esta abordagem manual garantiu um controlo granular sobre o ciclo de reconstrução dos widgets, mantendo o código leve.

5.4 UI Adaptável e Experiência de Utilizador (UI/UX)

Desafio: Garantir que a aplicação era visualmente consistente e funcional em diferentes ecrãs, sem conteúdos cortados por "notches" ou sobrepostos.

Solução: Utilização extensiva de widgets de layout adaptativo como o ***`SafeArea`*** (para respeitar as margens do sistema) e o *`SingleChildScrollView`* (para evitar erros de *overflow* em ecrãs pequenos). A adoção do **Material Design 3** através do *`ThemeData`* global assegurou uma coerência visual automática em toda a aplicação.

5.5 Fluidez e Performance (Assincronismo)

Desafio: Operações como a leitura do ficheiro JSON e pedidos de rede podiam bloquear a interface durante o arranque ou utilização.

Solução: Implementação rigorosa de programação assíncrona (`Future`, *`async`*/*`await`*). A inicialização do `DataManager` foi movida para o `main()` antes do `runApp`, garantindo que a aplicação só apresenta a interface quando os dados essenciais estão prontos, ou apresenta um estado de carregamento controlado (*`CircularProgressIndicator`*), evitando ecrãs brancos ou comportamentos imprevisíveis.

6 ANÁLISE CRÍTICA E MELHORIAS FUTURAS

A solução desenvolvida cumpre os requisitos propostos, destacando-se pela robustez e organização.

Melhorias Futuras:

- **Mapas:** Integração com Google Maps para visualização de localizações dos pontos de interesse num mapa interativo.
- **Internacionalização:** Suporte para múltiplos idiomas, permitindo a adaptação do conteúdo ao idioma do dispositivo do utilizador.
- **Backend de Dados:** Substituição dos assets locais por uma API remota, permitindo a atualização dinâmica de conteúdos (novos POIs, imagens) sem necessidade de atualizar a aplicação.
- **Geolocalização Dinâmica:** Implementação de serviços de localização (GPS) para detetar automaticamente a cidade do utilizador e apresentar conteúdo relevante para a sua localização atual.
- **Escalabilidade:** Expansão do dataset para incluir múltiplas cidades, tirando partido da estrutura de dados já implementada para suportar uma abrangência nacional/internacional.

7 CONCLUSÃO

O desenvolvimento deste guia turístico para a cidade de Coimbra permitiu a aplicação prática de conceitos avançados de engenharia de software no contexto de dispositivos móveis. O projeto cumpriu todos os requisitos funcionais e técnicos estabelecidos, resultando numa aplicação Android estável, reativa e visualmente apelativa.

A implementação técnica destacou-se pelo cumprimento rigoroso dos requisitos de persistência e navegação. A utilização de ***SharedPreferences*** garantiu que as preferências do utilizador (favoritos) fossem preservadas entre sessões, um aspeto crítico para a utilidade da aplicação. Paralelamente, o sistema de navegação baseado no ***Navigator*** nativo, enriquecido com uma gestão de histórico por abas, assegurou uma experiência de utilização fluida e intuitiva, respeitando os padrões de usabilidade da plataforma Android.

A integração bem-sucedida de APIs externas, a persistência de dados local e a implementação de uma navegação aninhada resiliente são os pilares técnicos que garantem a qualidade da experiência do utilizador final. Em suma, o trabalho realizado demonstra a aplicação prática de conhecimentos na plataforma Flutter e prepara o caminho para futuras expansões em direção a uma solução de guia turístico global e baseada na nuvem.



Instituto Superior
de Engenharia

Politécnico de Coimbra